Local Randomized Neural Networks Methods for Interface Problems

Yunlong Li and Fei Wang

1 Introduction

Interface problems appear in many complex physical situations such as multiphase flow and fluid-structure interactions where multiphysics coupling occurs across interfaces with potentially intricate geometries and dynamic boundaries. Traditional numerical methods often face challenges in handling the coupling conditions at these interfaces, especially when they have complex shapes or move over time.

Recently, neural network-based approaches have shown potential for solving partial differential equations (PDEs) with complex geometries, but they suffer from drawbacks including time-consuming optimization processes and the risk of getting trapped in local optima. Randomized Neural Networks (RNNs) offer an alternative approach by fixing randomly selected weights and biases in hidden layers, thereby avoiding expensive optimization solvers during training.

This paper proposes Local Randomized Neural Networks (LRNNs) method, a mesh-free approach where different RNNs are used to approximate solutions in different subdomains divided by interfaces. This method discretizes interface problems into linear systems at randomly sampled points and solves them using least-squares methods.

2 Model Interface Problems

2.1 Elliptic Interface Problem

Consider a domain Ω divided by an interface Γ into two subdomains Ω_1 and Ω_2. The elliptic interface problem is defined as:

$$-\nabla \cdot (\beta(x)\nabla u) = f, \qquad x \in \Omega, \tag{1}$$

$$[u] = g_1, \qquad x \in \Gamma, \tag{2}$$

$$[\beta(x)\nabla u \cdot n] = g_2, \qquad x \in \Gamma, \tag{3}$$

$$u = g_D, \qquad x \in \partial\Omega \tag{4}$$

where $\beta(x)$ is a piecewise positive constant function:

$$\beta(x) = \begin{cases} \beta_1, & x \in \Omega_1, \\ \beta_2, & x \in \Omega_2 \end{cases} \tag{5}$$

The notation $[u]$ denotes the jump of u across the interface, $[u] = u_1 - u_2$, and n is the unit outward normal vector on Γ from Ω_1 to Ω_2.

2.2 Parabolic Interface Problem

For time-dependent problems over interval $(0, T)$, the parabolic interface problem is:

$$\frac{\partial u}{\partial t} - \nabla \cdot (\beta(x)\nabla u) = f, \qquad (x,t) \in \Omega \times (0,T), \tag{6}$$

$$[u] = g_1, \qquad (x,t) \in \Gamma \times (0,T), \tag{7}$$

$$[\beta(x)\nabla u \cdot n] = g_2, \qquad (x,t) \in \Gamma \times (0,T), \tag{8}$$

$$u(x,t) = g(x,t), \qquad (x,t) \in \partial\Omega \times (0,T), \tag{9}$$

$$u(x,0) = u_0(x), \qquad x \in \Omega \tag{10}$$

Unlike the elliptic case, the interface Γ may be time-dependent, moving with a known motion. This can model scenarios like heat conduction in composite materials with dynamic interfaces.

3 Local Randomized Neural Networks Methods

3.1 Randomized Neural Networks

A fully connected neural network $\Psi : \mathbb{R}^{n_0} \to \mathbb{R}^{n_D}$ with depth D is defined as:

$$\Psi_0(x) = x, \tag{11}$$

$$\Psi_l(x) = \rho(W_l \Psi_{l-1} + b_l), \quad l = 1, \ldots, D-1, \tag{12}$$

$$\Psi = \Psi_D = W_D \Psi_{D-1} \tag{13}$$

Here, ρ is the activation function, and $W_l \in \mathbb{R}^{n_l \times n_{l-1}}$ and $b_l \in \mathbb{R}^{n_l}$ are the weights and biases in the l-th layer.

In a Randomized Neural Network (RNN), the weights and biases in the hidden layers are randomly generated and fixed, with only the weights of the output layer being adjustable. Any function $u_\rho \in \mathcal{N}_\rho^D$ (with one neuron in the output layer) can be expressed as a linear combination of basis functions:

$$u_\rho = \sum_{i=1}^{n_{D-1}} \alpha_i \psi_i \tag{14}$$

where ψ_i is the i-th component of the vector Ψ_{D-1}.

3.2 LRNNs Method for Elliptic Interface Problem

To solve the elliptic interface problem, the LRNNs method uses one RNN for each subdomain. For a domain split into two subdomains by interface Γ, two RNNs approximate the true solution in each subdomain:

$$u_\rho^1 = \sum_{k=1}^{m} \alpha_k^1 \psi_k^1, \quad u_\rho^2 = \sum_{k=1}^{m} \alpha_k^2 \psi_k^2 \tag{15}$$

Substituting these approximations into the elliptic interface problem and discretizing at randomly sampled collocation points (N_1 points in Ω, N_2 points on Γ, and N_3 points on $\partial\Omega$), we obtain a linear system:

$$AX = F, \tag{16}$$

$$BX = G_1, \tag{17}$$

$$CX = G_2, \tag{18}$$

$$DX = G_D \tag{19}$$

where A, B, C, D are matrices of order $N_1 \times 2m$, $N_2 \times 2m$, $N_2 \times 2m$, and $N_3 \times 2m$, respectively, and $X = (\alpha_1^1, \ldots, \alpha_m^1, \alpha_1^2, \ldots, \alpha_m^2)^T$ is the unknown vector.

The derivatives are approximated using finite difference schemes. For example, the first and second-order partial derivatives with respect to x are approximated as:

$$\frac{\partial u_\rho(x,y)}{\partial x} = \frac{u_\rho(x+h_x,y) - u_\rho(x-h_x,y)}{2h_x}, \tag{20}$$

$$\frac{\partial^2 u_\rho(x,y)}{\partial x^2} = \frac{u_\rho(x+h_x,y) - 2u_\rho(x,y) + u_\rho(x-h_x,y)}{h_x^2} \tag{21}$$

The weights of the RNNs' output layers are obtained by solving a least-squares problem with this linear system.

To account for the importance of different conditions, weighting factors γ_1, γ_2, and γ_3 can be introduced:

$$AX = F, \tag{22}$$

$$\gamma_1 BX = \gamma_1 G_1, \tag{23}$$

$$\gamma_2 CX = \gamma_2 G_2, \tag{24}$$

$$\gamma_3 DX = \gamma_3 G_D \tag{25}$$

These weights can be adjusted to emphasize the interface and boundary conditions, which are often crucial for accuracy.

3.3 Mixed LRNNs Method

By introducing another vector-valued function $p = \beta(x)\nabla u$, the elliptic interface problem can be rewritten in mixed form:

$$-\nabla \cdot p = f, \qquad x \in \Omega, \tag{26}$$

$$p - \beta(x)\nabla u = 0, \qquad x \in \Omega, \tag{27}$$

$$[u] = g_1, \qquad x \in \Gamma, \tag{28}$$

$$[p \cdot n] = g_2, \qquad x \in \Gamma, \tag{29}$$

$$u = g_D, \qquad x \in \partial\Omega \tag{30}$$

Additional RNNs are needed to approximate p:

$$u_\rho^1 = \sum_{k=1}^{m} \alpha_k^1 \psi_k^1, \quad u_\rho^2 = \sum_{k=1}^{m} \alpha_k^2 \psi_k^2, \tag{31}$$

$$p_\rho^1 = \left(\sum_{k=1}^{m} \tau_k^{1,1} \xi_k^{1,1}, \sum_{k=1}^{m} \tau_k^{1,2} \xi_k^{1,2} \right), \tag{32}$$

$$p_\rho^2 = \left(\sum_{k=1}^{m} \tau_k^{2,1} \xi_k^{2,1}, \sum_{k=1}^{m} \tau_k^{2,2} \xi_k^{2,2} \right) \tag{33}$$

Substituting these into the mixed form and adding weights γ_i, we get a larger linear system with matrices of order $N_1 \times 6m$, $N_2 \times 6m$, and $N_3 \times 6m$.

3.4 Space-time LRNNs Method for Parabolic Interface Problem

For parabolic interface problems, the LRNNs method uses a space-time approach where time and space variables are treated equally. The interface becomes a surface in the space-time domain, dividing it into two subdomains. Two RNNs approximate the solution:

$$u_\rho^1 = \sum_{k=1}^{m} \alpha_k^1 \psi_k^1, \quad u_\rho^2 = \sum_{k=1}^{m} \alpha_k^2 \psi_k^2 \tag{34}$$

This leads to a linear system:

$$AX = F, \tag{35}$$

$$\gamma_1 BX = \gamma_1 G_1, \tag{36}$$

$$\gamma_2 CX = \gamma_2 G_2, \tag{37}$$

$$\gamma_3 DX = \gamma_3 G_D, \tag{38}$$

$$\gamma_4 EX = \gamma_4 U_0 \tag{39}$$

where A, B, C, D, E are matrices of order $N_1 \times 2m$, $N_2 \times 2m$, $N_2 \times 2m$, $\frac{4N_3}{5} \times 2m$, and $\frac{N_3}{5} \times 2m$, respectively.

This space-time approach avoids time-stepping iteration and accumulation errors, solving the entire problem in one least-squares solution.

4 Numerical Examples and Results

4.1 Two-dimensional Flower Shape Interface Problem

A domain $\Omega = (-1, 1)^2$ with a flower-shaped interface:

$$\Gamma : (x-x_c)^2+(y-y_c)^2 = r^2(\theta), \quad r(\theta) = 0.4+0.2\sin(20\theta), \quad \theta \in [0, 2\pi) \tag{40}$$

The exact solution is:

$$u(x) = \begin{cases} \frac{1}{\beta_1} e^{xy}, & x \in \Omega_1, \\ \frac{1}{\beta_2} \sin(x)\sin(y), & x \in \Omega_2 \end{cases} \tag{41}$$

Two RNNs with tanh activation functions approximated the solution. Results showed:

- Error decreased with increasing neurons in hidden layers
- For $\beta_1 = 0.0001$ and $\beta_2 = 10000$, relative L^2 error reached 10^{-9}
- Method performed well even with diffusion coefficients having large variations
- Mixed LRNNs method achieved even smaller errors when sufficient sampling points were used

4.2 Three-dimensional Interface Problem

A domain $\Omega = (-1, 1)^3$ with a spherical interface:

$$\Gamma : x^2 + y^2 + z^2 = 0.75^2 \tag{42}$$

The exact solution:

$$u(x) = \begin{cases} 5e^{x^2+y^2+z^2} + 20, & x \in \Omega_1, \\ 10(x + y + z), & x \in \Omega_2 \end{cases} \tag{43}$$

Results showed:

- For $\beta_1 = 1$ and $\beta_2 = 1000$, relative L^2 error reached 10^{-5}
- Method was efficient, taking only about 6.8 seconds compared to 22.4 seconds for virtual element method
- Performance was robust across different diffusion coefficients

4.3 Multiple Interfaces and High-dimensional Problems

The method successfully handled:

- Problems with three interfaces, using four RNNs with relative L^2 error reaching 10^{-9}
- High-dimensional problems up to 20 dimensions with hyperplane interfaces
- Maintained accuracy even as dimension increased, with degrees of freedom not growing exponentially

4.4 Time-dependent Interface Problems

For parabolic problems with both fixed and moving interfaces, the space-time LRNNs approach showed:

- High accuracy with no error accumulation over time
- Relative L^2 error reaching 10^{-7} for problems with fixed interfaces
- Effective handling of dynamic interfaces with evolving geometry
- Robustness to different diffusion coefficients

5 Implications and Advantages

5.1 Computational Efficiency

The LRNNs method offers significant efficiency advantages:

- Avoids expensive optimization processes required by deep neural networks
- Solves linear least-squares problems instead of non-linear optimization
- Achieves high accuracy with fewer degrees of freedom
- Demonstrates faster computation times compared to traditional methods

5.2 Handling Complex Geometries

Being mesh-free, the method excels at problems with complex geometries:

- Eliminates the need for complex interface meshing and fitting
- Handles flower-shaped, spherical, and multiple interfaces effectively
- Adapts easily to dynamic interfaces that change over time
- Maintains accuracy even with nearly touching or intersecting interfaces

5.3 Solution Quality and Robustness

The LRNNs approach demonstrates several advantages in solution quality:

- Achieves spectral-like accuracy for smooth solutions
- Handles large variations in diffusion coefficients (ratios up to 10^{12})
- Space-time approach eliminates temporal error accumulation
- Performance improves predictably with increased sampling points and neurons

5.4 Scalability to Higher Dimensions

Unlike many traditional methods, LRNNs show good scalability:

- Successfully solves problems up to 20 dimensions
- Degrees of freedom scale with solution complexity rather than exponentially with dimension
- Maintains reasonable accuracy even in high dimensions
- Sampling strategy adapts naturally to higher-dimensional problems

The LRNNs method represents a significant advancement in solving interface problems, combining the flexibility of neural networks with the efficiency of randomized algorithms, while avoiding the optimization challenges of traditional deep learning approaches.